

TECHBIZ SECURITY

Cyber Security Internship Program

Week 3 – Task 9 (SOC)

Network Traffic Analysis – Wireshark Basics

Submitted By:	MANOJ
Role:	Cyber Security Intern
Internship Organisation:	TechBiz Security
Internship ID:	TBS-INT-2026-518
Submitted Date:	07 September 2026

Task Overview

This task focused on basic network traffic analysis using Wireshark, a packet-analysis tool widely used by Security Operations Center (SOC) analysts to inspect network communications, identify protocols in use, apply display filters, and investigate potentially suspicious traffic. The objective was to gain hands-on familiarity with packet capture, protocol filtering, and packet-level inspection as part of foundational SOC monitoring skills.

Task Objectives

1. Open and use Wireshark for network traffic analysis.
2. Capture and inspect network packets.
3. Understand packet structure and protocol information.
4. Apply Wireshark display filters.
5. Use the HTTP filter to identify HTTP traffic.
6. Use the DNS filter to identify DNS traffic.
7. Use the ip.addr filter to isolate traffic involving a specific IP address.
8. Inspect and analyze an HTTP request packet.
9. Understand how suspicious plaintext credentials or passwords could appear in unencrypted network traffic.
10. Document the practical work with screenshots.

1. Introduction to Wireshark

Wireshark is an open-source network protocol analyzer that captures and displays network traffic in a readable, packet-by-packet format. It allows an analyst to inspect the raw contents of network communication, including source and destination addresses, ports, protocols, and payload data.

For a SOC analyst, Wireshark is an important tool because it provides direct, packet-level visibility into what is actually happening on a network, rather than relying solely on summarized alerts. During incident investigation, packet analysis helps confirm whether an alert reflects genuine malicious activity, identify the protocols and hosts involved, and trace the sequence of communication between a source and a destination.

Using Wireshark, an analyst can identify active protocols, source and destination IP addresses, source and destination ports, and detailed packet fields, which together help in spotting unusual or suspicious communication patterns. Wireshark can be used to analyze traffic in two ways: capturing live traffic directly from a network interface, or opening a previously saved packet capture (PCAP/PCAPNG) file for offline analysis. This task used both approaches — traffic was captured live on the eth0 interface and reviewed using the resulting capture file.

2. Lab Environment

Parameter	Details
Tool	Wireshark
Platform	Kali Linux
Network Interface	eth0
Purpose	Network packet capture and analysis
Capture File	wireshark_eth07G8UU3.pcapng

3. Initial Packet Capture

Wireshark was opened on the eth0 interface, and live network traffic was captured and inspected. The initial capture view displayed a continuous stream of packets across several protocols, including TCP, TLSv1.2, and QUIC. The capture recorded a total of 2172 packets, with each entry showing the packet number, timestamp, source and destination address, protocol, packet length, and a summary of packet-level details in the Info column.

An initial, unfiltered packet capture is a useful starting point for a SOC analyst for the following reasons:

- Establishes visibility into the overall traffic passing through the monitored interface.
- Helps identify which protocols are actively in use on the network.
- Provides source and destination address information for further correlation.
- Provides a starting point from which deeper, filtered investigation can be carried out.

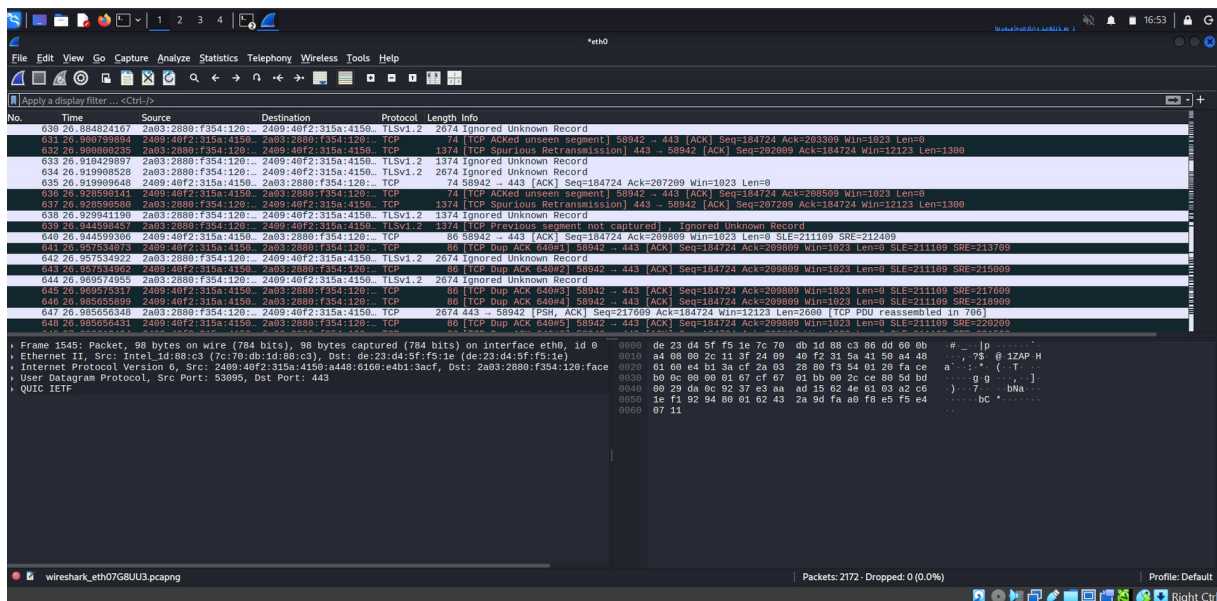


Figure 1: Initial Wireshark packet capture on the eth0 interface.

4. DNS Traffic Analysis

To isolate name-resolution traffic, the display filter dns was applied. DNS traffic represents the queries and responses exchanged when a client resolves a domain name to an IP address. DNS analysis is important in a SOC environment because domain resolution activity can reveal the destinations a host is attempting to reach, even before a connection is fully established.

The filtered capture showed DNS query and response packets exchanged between an internal host and a DNS resolver. The queries observed included lookups for domains such as chatgpt.com, web.whatsapp.com, docs.google.com, and ssl.gstatic.com. These domains are common, legitimate services and are not treated as malicious in this analysis; they are presented here purely as examples of the DNS activity observed during the capture.

DNS visibility gives a SOC analyst the ability to identify unusual or unexpected domains, unusual domain resolution patterns, possible command-and-control infrastructure, and unexpected destinations that may warrant further investigation. The Wireshark packet details pane was used to review both the DNS query and response fields for each selected packet.

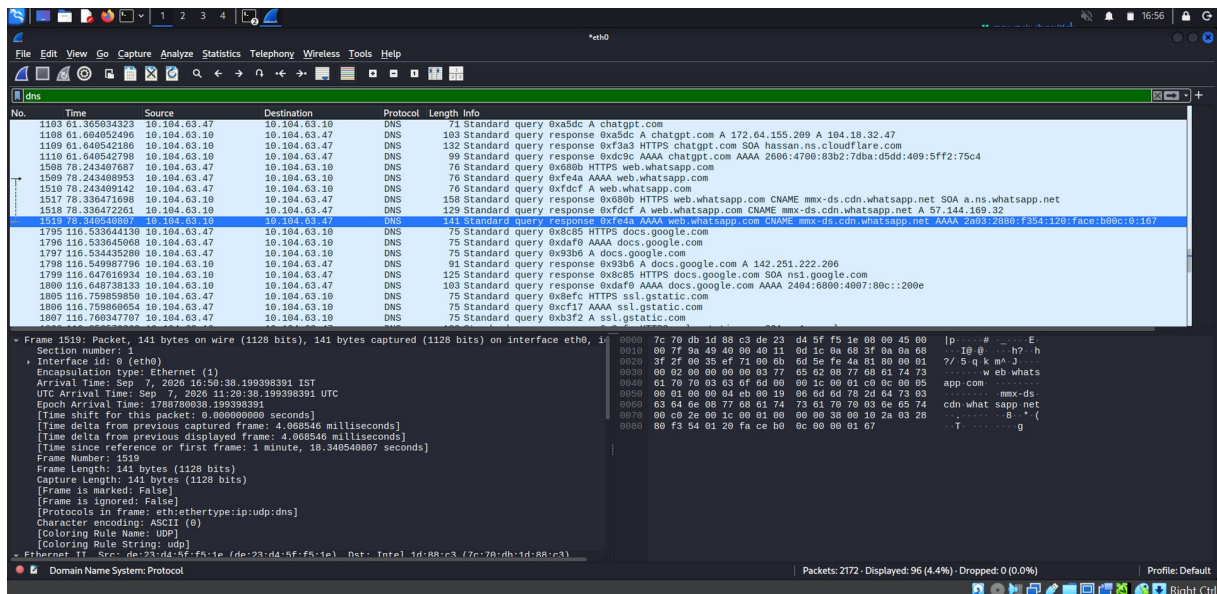


Figure 2: DNS traffic identified using the Wireshark dns display filter.

5. HTTP Traffic Analysis

The display filter http was applied to isolate HTTP traffic within the capture. This filter narrowed the results to two packets, representing a single HTTP request and its corresponding response.

Field	Value
Source	10.104.63.48
Destination	172.66.147.243
Protocol	HTTP
Request	GET / HTTP/1.1
Response	HTTP/1.1 200 OK

The GET request represents a client requesting a resource from the HTTP server, and the 200 OK response confirms that the server processed the request successfully and returned the requested content.

HTTP traffic is significant during security monitoring for several reasons:

- HTTP requests can expose request paths, headers, and other metadata.
- Because HTTP is unencrypted, it can expose sensitive information in transit.
- Analysts can inspect requests and responses directly for suspicious activity.
- HTTP traffic can reveal indicators such as unusual URLs, user agents, parameters, or transmitted data.

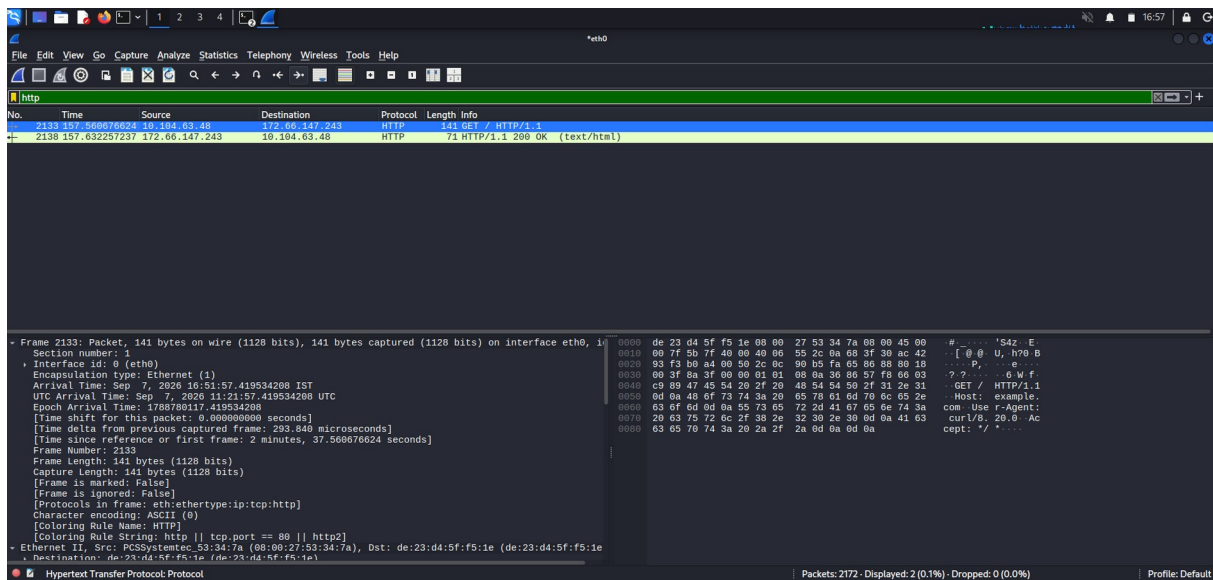


Figure 3: HTTP traffic identified using the Wireshark http display filter.

6. Analysis of One HTTP Request Packet

The HTTP GET request packet identified in the previous section was selected and inspected in detail using the Wireshark packet details pane. The following fields were reviewed:

Field	Value
Source IP	10.104.63.48
Destination IP	172.66.147.243
Protocol	HTTP
Request Method	GET
Requested Resource	/
HTTP Version	HTTP/1.1
Host	example.com
User-Agent	curl/8.20.0

Each of these fields provides specific investigative value:

- Source IP: identifies the system that generated the request.
- Destination IP: identifies the server receiving the request.
- HTTP Method (GET): indicates that a resource is being requested from the server.
- Requested Resource (/): represents the root resource of the target host.
- HTTP Version: identifies the version of the HTTP protocol used for the exchange.
- Host header: identifies the specific web host the client intended to reach.
- User-Agent: identifies the client software that generated the request.

This packet demonstrates how HTTP request metadata can be inspected directly in Wireshark. The screenshot below shows the packet details, and it does not contain any username or password field; it is provided solely to demonstrate HTTP header inspection.

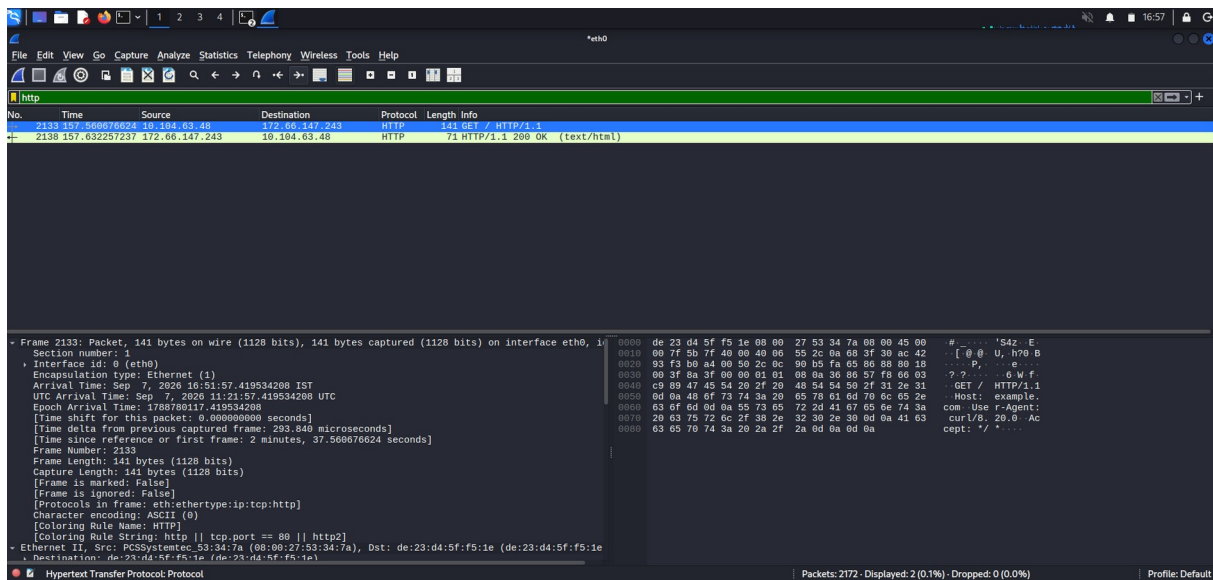


Figure 4: Detailed inspection of an HTTP GET request packet.

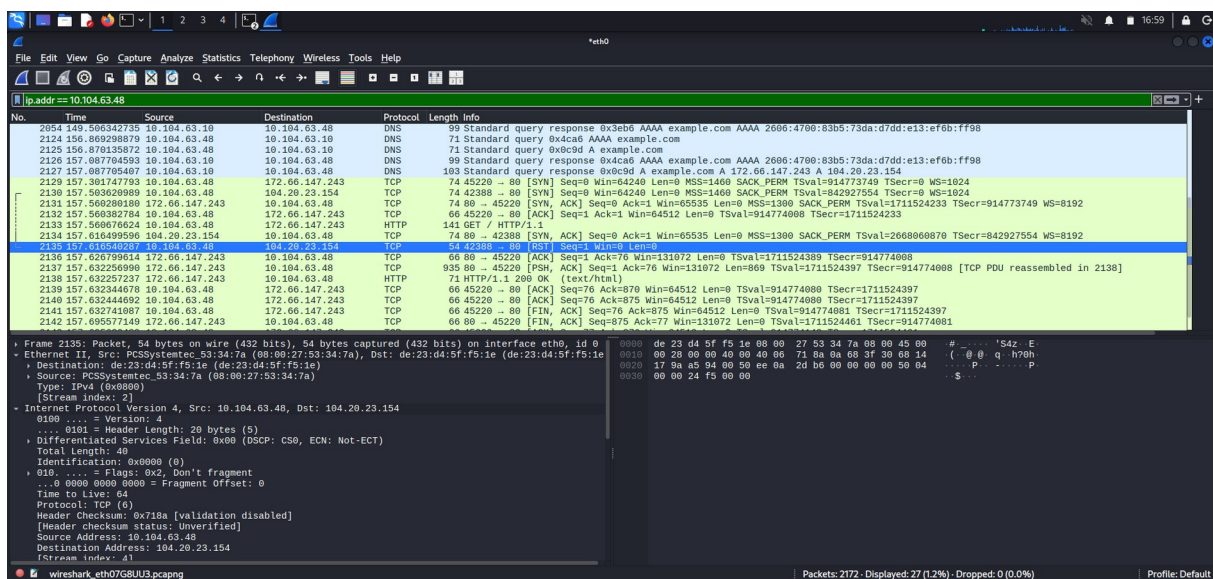
7. IP Address Display Filter

To focus the investigation on a single endpoint, the filter `ip.addr == 10.104.63.48` was applied. This filter displays all packets in which the specified address appears as either the source or the destination, regardless of the protocol involved.

The filtered results showed DNS, TCP, and HTTP traffic associated with the selected IP address, allowing the full sequence of communication involving that host to be reviewed in one place.

IP-based filtering is a useful technique for a SOC analyst investigating a particular endpoint or host referenced in an alert. It supports:

- Endpoint-focused investigation of a single host of interest.
- Identification of all communication partners for that host.
- Following a suspicious host across multiple protocols and sessions.
- Correlating DNS, TCP, and application-layer protocols for the same endpoint.
- Narrowing down large packet captures to relevant traffic only.

Figure 5: IP-based traffic filtering using `ip.addr == 10.104.63.48`.

8. Important Wireshark Display Filters Used

Filter	Purpose	Practical Use
dns	Displays DNS traffic	Analyze domain queries and responses
http	Displays HTTP traffic	Inspect web requests and responses
ip.addr == 10.104.63.48	Displays traffic involving the specified IP	Investigate a particular host

Display filters reduce the volume of visible traffic in a capture and allow an analyst to focus on a specific protocol, host, or conversation, which significantly speeds up investigation of large packet captures.

9. Understanding Suspicious Plaintext Password Traffic

Plaintext credentials can be exposed when an application transmits authentication information over an unencrypted protocol such as HTTP, FTP, or another insecure service. In an authorized security investigation, a SOC analyst may look for indicators such as HTTP POST requests containing login parameters, visible username fields, visible password fields, authentication data transmitted without encryption, and other sensitive information visible directly in packet payloads.

The security risk in this scenario is straightforward: if credentials are transmitted without encryption, an attacker capable of capturing that traffic on the network path may be able to read the credentials directly from the packet payload, without needing to break any encryption.

Relevant defensive controls include:

- Use HTTPS/TLS instead of HTTP for any traffic carrying credentials.
- Enforce encrypted communication across the environment.
- Avoid transmitting credentials over plaintext protocols.
- Monitor for unusual or unexpected authentication traffic.
- Use secure authentication mechanisms wherever possible.
- Apply network segmentation and monitoring where appropriate.

It is important to note that no plaintext password was captured or observed in the screenshots gathered for this task. This task required understanding how plaintext credentials could appear in unencrypted traffic; the HTTP screenshot obtained during this exercise demonstrates HTTP request inspection only, and does not contain an exposed password.

10. TryHackMe Requirement

The task referenced the TryHackMe room "Wireshark: The Basics" as a suggested practical resource. When this room was accessed, the page confirmed that it is a Premium room and requires a paid subscription to join, as shown in the screenshot below.

No paid or subscriber access was purchased for this task. In line with the instruction to avoid paid rooms and to document any limitation where a listed room is subscription-locked, the practical Wireshark exercises described in this report were instead performed locally using the Wireshark application installed on Kali Linux.

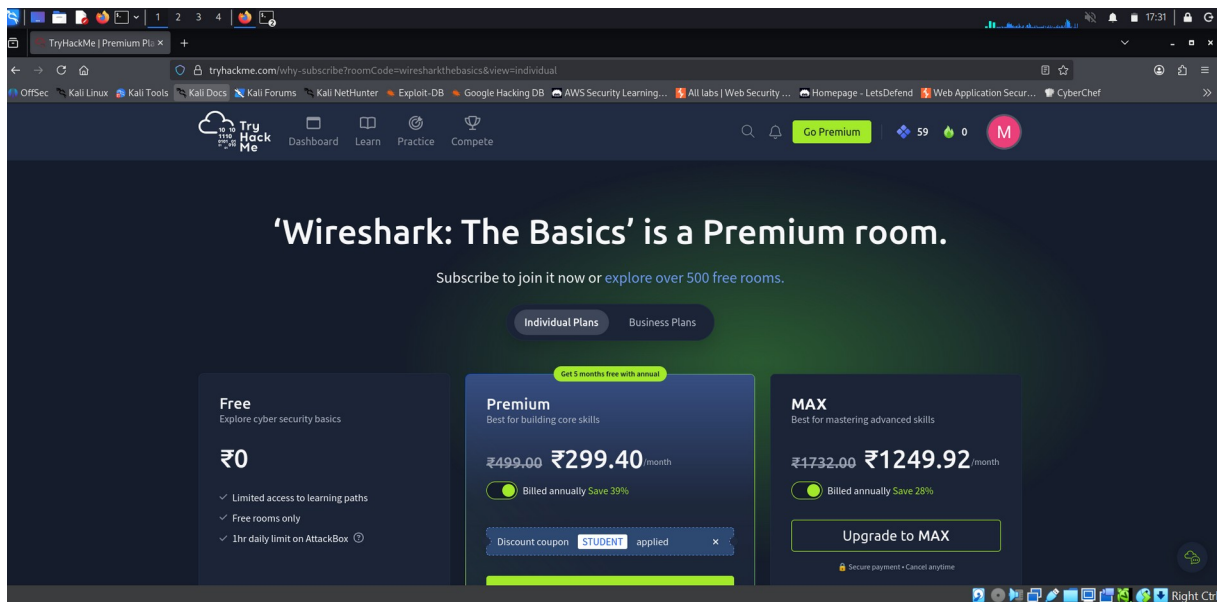


Figure 6: TryHackMe "Wireshark: The Basics" room showing Premium subscription requirement.

11. Practical Workflow

1. Opened Wireshark on Kali Linux.
2. Selected and used the eth0 interface for packet capture.
3. Inspected the captured network traffic.
4. Applied the dns display filter.
5. Reviewed DNS queries and responses.
6. Applied the http display filter.
7. Identified an HTTP GET request and the corresponding HTTP 200 OK response.
8. Selected and inspected the HTTP request packet in detail.
9. Applied the ip.addr == 10.104.63.48 filter.
10. Reviewed traffic associated with the selected IP address.
11. Documented the practical results using screenshots.

12. SOC Analyst Relevance

This practical exercise directly supports several core SOC responsibilities. Packet-level network traffic visibility allows an analyst to see what is actually occurring on the wire rather than relying only on summarized alerts. Protocol identification helps establish what services and communications are active on a monitored network. Host-based investigation, supported by IP address filtering, allows an analyst to trace all activity associated with a specific endpoint referenced in an alert.

DNS monitoring and HTTP traffic inspection provide insight into the domains a host is contacting and the web requests it is making, both of which are common areas of interest during malware or phishing investigations. Together, these skills support the identification of potentially suspicious network behavior and provide the packet-level evidence needed to support incident-response investigations. Wireshark enables an analyst to move from a high-level alert to concrete, packet-level evidence, which is a fundamental capability within SOC operations.

13. Key Learnings

- Learned the basic Wireshark interface and packet list view.
- Learned how to inspect captured network traffic.

- Learned the dns display filter.
- Learned the http display filter.
- Learned the ip.addr display filter.
- Learned how to inspect an HTTP GET request in detail.
- Understood how to interpret source and destination IP addresses.
- Understood the security risks associated with plaintext protocols.
- Learned how packet-level analysis supports SOC investigations.
- Learned the importance of narrowing large packet captures using display filters.

14. Challenges and Resolution

Challenge

The assigned TryHackMe room "Wireshark: The Basics" was not accessible because it was displayed as a Premium room requiring a paid subscription.

Resolution

The Premium subscription was not purchased. Instead, the required Wireshark concepts and practical packet-analysis activities were performed locally using Wireshark on Kali Linux, with screenshots captured as evidence to document each stage of the exercise.

15. Conclusion

This task provided practical exposure to basic network traffic analysis using Wireshark. The exercise covered packet capture inspection, DNS traffic filtering and analysis, HTTP traffic filtering and request analysis, IP-based traffic filtering, and the security implications of plaintext communication. As the assigned TryHackMe room was subscription-locked under the current Free plan, the equivalent practical work was carried out locally using Wireshark, and each step was documented with corresponding screenshots.

The skills exercised in this task — traffic capture, protocol filtering, DNS analysis, HTTP inspection, and endpoint-focused filtering — are directly relevant to SOC monitoring, network investigation, alert validation, and incident response.

16. Evidence Summary

#	Screenshot	Description
1	01_wireshark_initial_packet_capture.png	Initial packet capture on eth0
2	02_wireshark_dns_filter.png	DNS traffic filtered using dns
3	03_wireshark_http_filter.png	HTTP traffic and HTTP GET request analysis
4	04_wireshark_ip_addr_filter.png	IP-based filtering using ip.addr
5	06_tryhackme_wireshark_the_basics_premium_notice.png	TryHackMe Premium room limitation